

LAB GUIDE WITH SOLUTIONS



Module 1: Introduction to Databases & RDBMS Fundamentals

Module 1 - 6 Hours

Introduction to Databases & RDBMS Fundamentals

How to use this lab guide

There are six labs, sized for a three-hour session. Every lab states an objective, the exact steps, the expected output, and a short explanation of what just happened. **Solutions are included inline** - this is the guided lab, not the assessment. The unguided practice is in [M01_Exercises.pdf](#).

Lab	Title	Time
1.1	Install and verify PostgreSQL	25 min
1.2	Install Oracle XE and MySQL	35 min
1.3	Create the ICICI Bank database	20 min
1.4	Explore the database architecture	30 min
1.5	First CRUD operations	30 min
1.6	Demonstrate ACID	40 min

Before you start: you need administrator rights on your machine, roughly 15 GB of free disk, and a text editor. Keep a file called `module1_notes.sql` open and paste every statement you run into it. You will be asked for it at the end of the day.

Lab 1.1 - Install and verify PostgreSQL

Objective: get a running PostgreSQL 18 server and connect to it two different ways.

Steps

1. Download the PostgreSQL 18 Windows installer from postgresql.org/download/windows. Run it and accept the defaults, with two exceptions:

- Set the `postgres` superuser password to something you will remember. Write it down. Use `ICICBank@2026` in the classroom so everyone matches.
- Keep the port as **5432**.

2. The installer offers Stack Builder at the end. Skip it.

3. Confirm the service is running.

```
Get-Service -Name postgresql*
```

Expected:

Status	Name	DisplayName
Running	postgresql-x64-18	postgresql-x64-18 - PostgreSQL Server 18

4. Connect with the command-line client. Add PostgreSQL's `bin` folder to your PATH first, or use the full path.

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -h localhost
```

Expected:

```
Password for user postgres:
psql (18.x)
Type "help" for help.

postgres=#
```

5. Ask the server what it is.

```
SELECT version();
SELECT current_database(), current_user, inet_server_port();
```

Expected output resembles:

```
              version
-----
PostgreSQL 18.2 on x86_64-windows, compiled by msvc-19...

current_database | current_user | inet_server_port
-----+-----+-----
postgres        | postgres    |          5432
```

6. Open pgAdmin from the Start menu, register the local server, and browse to [Databases](#) > [postgres](#) > [Schemas](#) > [public](#). You are looking at the same server through a second client.

What just happened

You started a **database server process** listening on TCP port 5432, and connected to it twice - once from [psql](#), once from pgAdmin. Neither client opened a data file. Both sent SQL over a socket and received rows back. That is the client-server architecture from section 5.1 of the course material, running on your laptop.

If the connection is refused: the service is not running, or a firewall is blocking 5432, or you typed the password wrong. Check the service first; that is the cause about eight times out of ten.

Lab 1.2 - Install Oracle XE and MySQL

Objective: see the same concepts under three different engines, and notice what changes.

Part A - Oracle Database XE 21c

1. Download Oracle Database 21c Express Edition for Windows from Oracle's site (a free Oracle account is required). Run `setup.exe`.
2. Set the `SYS / SYSTEM` password to `ICICIBank@2026`. Note the listener port, **1521**.
3. After install, connect:

```
sqlplus system/ICICIBank@2026@localhost:1521/XEPDB1
```

4. Ask Oracle what it is.

```
SELECT banner FROM v$version;  
SELECT name, open_mode FROM v$database;  
SELECT sys_context('USERENV','CON_NAME') AS container FROM dual;
```

Expected:

```
BANNER  
-----  
Oracle Database 21c Express Edition Release 21.0.0.0.0  
  
NAME      OPEN_MODE  
-----  
XE        READ WRITE  
  
CONTAINER  
-----  
XEPDB1
```

Note the word `dual`. Oracle requires a FROM clause on every SELECT, so it provides a one-row dummy table called `dual`. PostgreSQL, MySQL and SQL Server let you write `SELECT 1;` with no FROM at all. This is the first of many dialect differences you will meet.

Part B - MySQL 8

1. Install MySQL 8 Community Server with MySQL Workbench. Root password `ICICIBank@2026`, port **3306**.
2. Connect and interrogate it.

```
mysql -u root -p
```

```
SELECT VERSION();  
SHOW DATABASES;  
SELECT @@default_storage_engine;
```

Expected:

```
+-----+
| VERSION() |
+-----+
| 8.0.36 |
+-----+

+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+

+-----+
| @@default_storage_engine |
+-----+
| InnoDB |
+-----+
```

Part C - Compare what you saw

Fill this in from your own three sessions.

	PostgreSQL	Oracle XE	MySQL
Default port	5432	1521	3306
Command-line client	<code>psql</code>	<code>sqlplus</code>	<code>mysql</code>
A "database" means	A database inside a cluster	A pluggable database inside a container	A schema
A "schema" means	A namespace inside a database	A user's set of objects	Same thing as a database
Dummy table needed for <code>SELECT 1</code>	No	Yes, <code>dual</code>	No
List tables	<code>\dt</code>	<code>SELECT table_name FROM user_tables;</code>	<code>SHOW TABLES;</code>

What just happened

Three engines, three vocabularies, one relational model. The most common source of confusion for a new data engineer is that **"schema" and "database" mean different things in each product**. In Oracle a schema is a user. In MySQL a schema is a database. In PostgreSQL a database contains many schemas. Write that down; it causes real bugs during the migration work in Module 11.

Lab 1.3 - Create the ICICI Bank database

Objective: load the schema and dataset that every remaining module depends on.

Steps

1. Locate the `sql` folder in `00_Programme_Overview` beside this document. It contains the schema in four dialects and the seed data.
2. Create the database. From a Windows terminal:

```
& "C:\Program Files\PostgreSQL\18\bin\createdb.exe" -U postgres icicibank
```

Or, from inside `psql`:

```
CREATE DATABASE icicibank;
```

3. Load the schema, then the data.

```
psql -U postgres -d icicibank -f "00_Programme_Overview\sql\01_schema_postgres.sql"
psql -U postgres -d icicibank -f "00_Programme_Overview\sql\02_seed_postgres.sql"
```

The schema file prints `NOTICE: table "... does not exist, skipping` for each DROP on a fresh database. That is expected and harmless.

4. Verify the load. This is the single most important check of the day.

```
\c icicibank

SELECT 'branch' AS table_name, COUNT(*) AS row_count FROM branch
UNION ALL SELECT 'customer',      COUNT(*) FROM customer
UNION ALL SELECT 'account',      COUNT(*) FROM account
UNION ALL SELECT 'account_txn',  COUNT(*) FROM account_txn
UNION ALL SELECT 'loan',         COUNT(*) FROM loan
UNION ALL SELECT 'loan_payment', COUNT(*) FROM loan_payment
UNION ALL SELECT 'card',         COUNT(*) FROM card
UNION ALL SELECT 'beneficiary',  COUNT(*) FROM beneficiary
UNION ALL SELECT 'employee',     COUNT(*) FROM employee
ORDER BY table_name;
```

Expected:

table_name	row_count
account	36
account_txn	495
beneficiary	60
branch	8
card	32
customer	30
employee	24
loan	12
loan_payment	144

If your numbers are higher, you ran the seed file twice. Drop the database and repeat from step 2. Do not try to delete the extra rows by hand - foreign keys will fight you, and you will learn why in Module 4.

5. Look at real data for the first time.

```
SELECT customer_id, first_name, last_name, city, kyc_status
FROM   customer
ORDER  BY customer_id
LIMIT  5;
```

Expected:

customer_id	first_name	last_name	city	kyc_status
1001	Aarav	Sharma	Mumbai	REJECTED
1002	Diya	Nair	Mumbai	VERIFIED
1003	Rohan	Iyer	Bengaluru	VERIFIED
1004	Meera	Reddy	Bengaluru	VERIFIED
1005	Karthik	Gupta	Chennai	VERIFIED

What just happened

You created a database, defined nine tables with primary keys, foreign keys and check constraints, and loaded about 840 rows of synthetic banking data. The dataset is generated deterministically, so every learner in the room has identical data and identical query results. That matters from Module 3 onward, when answers become numbers you can compare.

Lab 1.4 - Explore the database architecture

Objective: see the architecture concepts from section 5 as real, queryable objects.

Step 1 - Databases, schemas and tables

```
\1
```

Expected: a list including `icicibank`, `postgres`, `template0`, `template1`.

```
\dn
\dt
```

`\dn` shows the `public` schema. `\dt` lists the nine ICICI Bank tables. Note the layering: **cluster** > **database** > **schema** > **table**.

Step 2 - Look at the catalog itself

The database describes itself in tables. This is Codd's information rule in action.

```
SELECT table_name, column_name, data_type, is_nullable
FROM   information_schema.columns
WHERE  table_name = 'account'
ORDER BY ordinal_position;
```

Expected:

table_name	column_name	data_type	is_nullable
account	account_id	integer	NO
account	account_number	character	NO
account	customer_id	integer	NO
account	branch_id	integer	NO
account	account_type	character varying	NO
account	balance	numeric	NO
account	currency	character	NO
account	status	character varying	NO
account	opened_date	date	NO
account	interest_rate	numeric	YES

Step 3 - See the declared relationships

```
SELECT tc.table_name,
       kcu.column_name,
       ccu.table_name AS references_table,
       ccu.column_name AS references_column
FROM   information_schema.table_constraints tc
JOIN   information_schema.key_column_usage kcu
       ON tc.constraint_name = kcu.constraint_name
JOIN   information_schema.constraint_column_usage ccu
       ON ccu.constraint_name = tc.constraint_name
```

```
WHERE tc.constraint_type = 'FOREIGN KEY'
ORDER BY tc.table_name;
```

Expected: ten foreign keys, including `account.customer_id -> customer.customer_id` and `employee.manager_id -> employee.employee_id`.

The `employee` row is worth pausing on. A table can reference **itself**. That is how a manager hierarchy is stored, and you will query it with a self join in Module 3.

Step 4 - Physical size

```
SELECT relname AS table_name,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM   pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

Expected: `account_txn` is the largest table, at roughly 100 to 150 KB including its indexes. In production it would be the largest by a factor of thousands.

Step 5 - Memory configuration

```
SHOW shared_buffers;
SHOW work_mem;
SHOW wal_level;
SHOW max_connections;
```

Typical defaults:

```
shared_buffers | 128MB
work_mem       | 4MB
wal_level      | replica
max_connections| 100
```

`shared_buffers` is the buffer cache from section 5.4. `wal_level = replica` means the write-ahead log carries enough information to feed a standby server, which is what Module 9 builds.

Step 6 - Who is connected

```
SELECT pid, username, application_name, client_addr, state
FROM   pg_stat_activity
WHERE  datname = 'icicibank';
```

You will see your own session. Open a second `psql` window and re-run it to see two.

What just happened

Every architectural concept in section 5 turned out to be inspectable with SQL. The catalog is just tables. The buffer cache is a setting you can read. Connections are rows. When you get to Module 9 monitoring and Module 10 administration, this is the skill you will use constantly: **ask the database about itself**.

Lab 1.5 - First CRUD operations

Objective: perform Create, Read, Update and Delete against real banking tables.

Work inside a transaction so you can undo everything at the end. You have not been taught transactions properly yet - that is Lab 1.6 - so for now just follow the steps.

C - Create

ICICI Bank opens a new savings account for a walk-in customer at the Fort Mumbai branch.

```
INSERT INTO customer
(customer_id, first_name, last_name, date_of_birth, email, phone,
city, kyc_status, risk_rating, customer_since, home_branch_id)
VALUES
(1031, 'Vinod', 'Menon', DATE '1991-04-17', 'vinod.menon@example.com',
'9845012345', 'Mumbai', 'VERIFIED', 'LOW', DATE '2026-02-10', 1);

INSERT INTO account
(account_id, account_number, customer_id, branch_id, account_type,
balance, currency, status, opened_date, interest_rate)
VALUES
(9046, 'ICIC0000009046', 1031, 1, 'SAVINGS',
25000.00, 'INR', 'ACTIVE', DATE '2026-02-10', 3.50);
```

Expected: `INSERT 0 1` twice.

Now try it in the wrong order. Roll back first, then attempt the account insert before the customer exists:

```
INSERT INTO account
(account_id, account_number, customer_id, branch_id, account_type,
balance, currency, status, opened_date, interest_rate)
VALUES
(9047, 'ICIC0000009047', 9999, 1, 'SAVINGS',
1000.00, 'INR', 'ACTIVE', DATE '2026-02-10', 3.50);
```

Expected:

```
ERROR: insert or update on table "account" violates foreign key constraint
"fk_account_customer_id"
DETAIL: Key (customer_id)=(9999) is not present in table "customer".
```

That error message is the point of the lab. The database refused to create an account for a customer who does not exist. No application code was involved. This is referential integrity, and it is a preview of Module 4.

R - Read

```
-- The new customer's account
SELECT a.account_number, a.account_type, a.balance, a.status
FROM   account a
WHERE  a.customer_id = 1031;
```

```
-- All active savings accounts with more than 100,000
SELECT account_number, balance
FROM   account
WHERE  account_type = 'SAVINGS'
      AND status = 'ACTIVE'
      AND balance > 100000
ORDER BY balance DESC;

-- How many accounts does each branch hold?
SELECT branch_id, COUNT(*) AS accounts, SUM(balance) AS total_balance
FROM   account
GROUP BY branch_id
ORDER BY branch_id;
```

U - Update

The customer deposits 15,000 in cash at the branch counter.

```
UPDATE account
SET   balance = balance + 15000
WHERE account_id = 9046;

SELECT account_number, balance FROM account WHERE account_id = 9046;
```

Expected: balance is now **40000.00**.

Note `balance = balance + 15000` rather than `balance = 40000`. Always compute from the current value. Writing an absolute figure means you overwrite any change another session made in between, which is the "lost update" problem from section 6.4.

D - Delete

```
DELETE FROM account WHERE account_id = 9046;
DELETE FROM customer WHERE customer_id = 1031;
```

Order matters. Delete the account first. Try the reverse and you get:

```
ERROR: update or delete on table "customer" violates foreign key constraint
       "fk_account_customer_id" on table "account"
DETAIL: Key (customer_id)=(1031) is still referenced from table "account".
```

Clean up

```
SELECT COUNT(*) FROM customer; -- must be 30 again
SELECT COUNT(*) FROM account;  -- must be 36 again
```

What just happened

You exercised all four data manipulation operations, and twice the database stopped you from doing something that would have corrupted the bank's data. In a file-based system both of those mistakes would have succeeded silently. That is the answer to ticket ICICI-4417.

Lab 1.6 - Demonstrate ACID

Objective: prove each of the four properties yourself, rather than taking them on trust.

You need **two psql sessions side by side**. Open two terminals, connect both to `icicibank`, and label them Session A and Session B.

A - Atomicity

Session A:

```
SELECT account_id, balance FROM account WHERE account_id IN (9001, 9007);
```

Write both balances on paper.

```
BEGIN;
UPDATE account SET balance = balance - 25000 WHERE account_id = 9001;
UPDATE account SET balance = balance + 25000 WHERE account_id = 9007;
SELECT account_id, balance FROM account WHERE account_id IN (9001, 9007);
```

Both balances have changed. Now abandon the transfer:

```
ROLLBACK;

SELECT account_id, balance FROM account WHERE account_id IN (9001, 9007);
```

Both balances are back to the numbers on your paper. **Neither update survived, because a transaction is all or nothing.**

Now do it again and commit:

```
BEGIN;
UPDATE account SET balance = balance - 25000 WHERE account_id = 9001;
UPDATE account SET balance = balance + 25000 WHERE account_id = 9007;
COMMIT;

SELECT account_id, balance FROM account WHERE account_id IN (9001, 9007);
```

Both changed together. Restore the original state before continuing:

```
UPDATE account SET balance = balance + 25000 WHERE account_id = 9001;
UPDATE account SET balance = balance - 25000 WHERE account_id = 9007;
```

C - Consistency

The schema already carries `ck_account_3 CHECK (balance >= 0)`. Test it.

```
SELECT balance FROM account WHERE account_id = 9001;

UPDATE account SET balance = balance - 99999999 WHERE account_id = 9001;
```

Expected:

```
ERROR: new row for relation "account" violates check constraint "ck_account_3"  
DETAIL: Failing row contains (9001, ICIC0000009001, ..., -99821550.00, ...).
```

Try to break referential integrity too:

```
INSERT INTO account_txn  
  (txn_id, account_id, txn_date, txn_type, amount, channel, narration, status, reference_no)  
VALUES  
  (999999, 8888, TIMESTAMP '2026-03-01 10:00:00', 'DEBIT', 500, 'ATM', 'test', 'SUCCESS', 'REF999');
```

Expected: a foreign key violation on `fk_account_txn_account_id`. There is no account 8888, so there can be no transaction on it.

And an invalid domain value:

```
INSERT INTO account_txn  
  (txn_id, account_id, txn_date, txn_type, amount, channel, narration, status, reference_no)  
VALUES  
  (999999, 9001, TIMESTAMP '2026-03-01 10:00:00', 'TRANSFER', 500, 'ATM', 'test', 'SUCCESS', 'REF999');
```

Expected: a check constraint violation, because `txn_type` may only be `DEBIT` or `CREDIT`.

Three attempts to write invalid banking data, three refusals, zero application code.

I - Isolation

Session A:

```
BEGIN;  
UPDATE account SET balance = balance - 5000 WHERE account_id = 9002;  
SELECT balance FROM account WHERE account_id = 9002;  
-- do NOT commit yet
```

Session B:

```
SELECT balance FROM account WHERE account_id = 9002;
```

Session B sees the **old** balance. Session A's uncommitted change is invisible to it. That is isolation: no dirty reads.

Session B, now try to change the same row:

```
UPDATE account SET balance = balance - 1000 WHERE account_id = 9002;
```

Session B **hangs**. It is waiting for Session A's row lock.

Session A:

```
ROLLBACK;
```

Session B immediately completes. Roll it back too:

```
ROLLBACK;
```

What you just observed: writers block writers on the same row, but readers are never blocked. PostgreSQL achieves this with MVCC, keeping an older version of the row available for readers. Module 6 goes into the mechanism, the isolation levels, and what happens when two sessions block each other in a cycle - a deadlock.

D - Durability

```
BEGIN;  
UPDATE account SET balance = balance + 1 WHERE account_id = 9001;  
COMMIT;
```

Now kill the server hard. From an elevated PowerShell:

```
Stop-Service postgresql-x64-18 -Force  
Start-Service postgresql-x64-18
```

Reconnect and check:

```
SELECT balance FROM account WHERE account_id = 9001;
```

The extra rupee is still there. The commit was durable across a full service restart because the WAL record reached disk before `COMMIT` returned.

Look at what made that possible:

```
SELECT pg_current_wal_lsn();  
SHOW wal_level;  
SHOW fsync;
```

`fsync = on` means PostgreSQL forces the log to physical disk before acknowledging a commit. Turning it off makes writes much faster and makes durability a lie. **Never turn it off on a system holding money.**

Put the rupee back:

```
UPDATE account SET balance = balance - 1 WHERE account_id = 9001;
```

What just happened

You demonstrated all four ACID properties in about forty minutes:

Property	How you proved it
Atomicity	A rolled-back transfer left both balances untouched
Consistency	Three invalid writes were refused by declared constraints
Isolation	Session B could not see or overwrite Session A's uncommitted row
Durability	A committed change survived killing the database service

End-of-module checklist

Tick every line before you leave. Modules 2 to 12 assume all of it.

	Item
☐	PostgreSQL 18 is running and I can connect with <code>psql</code> and pgAdmin
☐	Oracle XE is running and I can connect with <code>sqlplus</code>
☐	MySQL 8 is running and I can connect with <code>mysql</code>
☐	The <code>icicibank</code> database exists with 9 tables and the expected row counts
☐	I can list databases, schemas, tables and columns from the catalog
☐	I have run INSERT, SELECT, UPDATE and DELETE against <code>customer</code> and <code>account</code>
☐	I have seen a foreign key violation and a check constraint violation
☐	I demonstrated atomicity, consistency, isolation and durability
☐	My <code>module1_notes.sql</code> file contains every statement I ran

Common problems and fixes

Symptom	Cause	Fix
<code>psql: could not connect to server</code>	Service stopped	<code>Start-Service postgresql-x64-18</code>
<code>password authentication failed</code>	Wrong password, or connecting as the wrong user	Re-run with <code>-U postgres</code>
Row counts are double the expected	Seed file run twice	<code>DROP DATABASE icicibank;</code> and reload
<code>relation "account" does not exist</code>	Connected to the wrong database	<code>\c icicibank</code>
Oracle: <code>ORA-12541 TNS no listener</code>	Listener service stopped	<code>Start OracleOraDB21Home1TNSListener</code>
MySQL: tables show <code>MyISAM</code>	Wrong default engine	<code>ALTER TABLE x ENGINE=InnoDB;</code> and fix <code>my.ini</code>
Session hangs forever on UPDATE	Another session holds the row lock	Commit or roll back the other session